

OmniDraft

From rough idea to polished draft — one workspace.

Project Report & Concept Note

IBM SkillsBuild — Gen AI & Cloud Computing 2026

Vibe Coding Masterclass Series

Author: Amal Chandran

Date: July 2026

Table of Contents

1. Project Concept Note
2. Application Overview & Tech Stack
3. Prompting Strategy & Frameworks
4. Phase-by-Phase Development Summary
5. Application Architecture
6. Challenges & Resolutions
7. Key Learnings & Reflection
8. Appendix: Sample Prompts

1. Project Concept Note

1.1 Project Title

OmniDraft — AI-Powered Content Creation Workspace

1.2 Problem Statement / Objective

Professionals and creators routinely juggle multiple tools — one for email drafting, another for summarization, a third for creative writing. This context-switching fragments workflow, reduces productivity, and increases cognitive load. Existing AI writing tools are often single-purpose (e.g., chat-only or summary-only), requiring users to leave one environment for another.

OmniDraft solves this by unifying three core content modes — drafting, summarization, and creative writing — into a single, streamlined workspace powered by a large language model. Users can switch modes instantly, access pre-built templates, upload files, stream AI responses in real time, and persist their conversation history — all without leaving the application.

1.3 Target User & Use Case

User Persona	Use Case
Business professionals	Draft polished emails, proposals, and reports
Students & researchers	Summarize articles, papers, and lecture notes
Content creators & writers	Generate blog posts, social captions, and stories
Busy professionals	Get TL;DR summaries of long documents

The primary audience is anyone who regularly produces or consumes text content and wants a single AI-powered assistant to draft, summarize, and create — faster and with less tool-switching.

1.4 LLM Model & API

- **Primary Model:** NVIDIA Nemotron (z-ai/glm-5.2) via NVIDIA Build API

- **Fallback Model:** Meta Llama 3.1 8B Instruct (meta/llama-3.1-8b-instruct)
- **API Type:** NVIDIA Cloud API — Server-Sent Events (SSE) streaming for real-time token delivery
- **Integration:** httpx async client with 30-second read timeout, retry logic, and mode-specific system prompts

1.5 Key Features

Feature	Description
Three-Mode Wheel	Switch instantly between Draft, Summarize, and Creative modes
Real-time SSE Streaming	Token-by-token response rendering via Server-Sent Events
Pre-built Templates	12 curated prompts (4 per mode) for common tasks
File Upload	Upload PDF/TXT files for AI summarization (Summarize mode)
Conversation History	Persisted sessions per user in PostgreSQL
Magic Link Auth	Passwordless authentication via Supabase Auth
Export & Copy	Download conversations as TXT or Markdown; one-click clipboard copy
Dark Theme	OLED-optimized dark interface with glassmorphic design

1.6 Expected User Experience & Outcomes

Users land on a polished marketing page with an interactive mode wheel. After signing in with a single email link (no password), they enter a workspace where they select a mode, optionally choose a template, type their prompt, and watch the AI response stream in word by word. Conversations are automatically saved and accessible from a sidebar. The expected outcomes are:

- Reduced time spent switching between writing tools
- Faster first drafts via AI-assisted generation
- Improved comprehension through instant summarization
- Creative inspiration through structured prompts and templates
- A seamless, enjoyable user experience with zero configuration

2. Application Overview & Tech Stack

2.1 Application Overview

OmniDraft is a full-stack, single-page web application consisting of a React frontend, a FastAPI backend, and a Supabase-managed PostgreSQL database. The application is containerized with Docker, served via Nginx reverse proxy, and deployed to AWS Elastic Beanstalk. The frontend communicates with the backend over HTTP (to be upgraded to HTTPS), and the backend in turn communicates with NVIDIA's LLM API over SSE streaming for real-time token generation.

2.2 Tech Stack

Layer	Technology	Version	Purpose
Frontend	React	18.3	UI framework
	TypeScript	5.5	Type safety
	Vite	5.4	Build tool & dev server
	Tailwind CSS	3.4	Utility-first styling
	Framer Motion	11	Animations
	Zustand	4.5	State management
	React Router	6.26	Client-side routing
	Supabase JS	2.45	Auth & DB client
Backend	Python FastAPI	0.115	Web framework
	Uvicorn	0.30	ASGI server
	httpx	0.27	Async HTTP client
	Pydantic	2	Data validation
	slowapi	0.1	Rate limiting
	Supabase Python	2.5	DB service client
AI / LLM	NVIDIA Nemotron API	—	LLM inference (SSE streaming)
	Fallback: Llama 3.1 8B	—	Backup model

Database	Supabase PostgreSQL	—	4 tables + RLS
Auth	Supabase Auth	—	Magic link passwordless
Storage	Supabase Storage	—	File uploads (S3-compatible)
Container	Docker	—	Multi-stage build
Proxy	Nginx	—	Reverse proxy + static files
Cloud	AWS Elastic Beanstalk	—	Deployment (t3.micro)
	Amazon ECR	—	Container registry

3. Prompting Strategy & Frameworks

3.1 Vibe Coding Methodology

OmniDraft was built entirely through AI-assisted development — "Vibe Coding" as coined by Andrej Karpathy. The development process followed a structured prompt engineering methodology that combined several frameworks:

3.1.1 Role-Goal-Context (RGC) Framework

Every prompt began with a clear role definition, a specific goal, and a rich context block. This eliminated ambiguity and produced high-quality first-pass code.

Pattern: "Act as a [Role]. Build [Goal] using [Tech Stack] with [Constraints]. The user is [Context]."

3.1.2 Markdown-Based Specification Sheets

Rather than natural-language descriptions, each major component was specified using a structured Markdown template with sections for Vibe Profile, Product Brief, Tech Rails, and Design Tokens. This gave the AI unambiguous constraints, reducing hallucinated code by an estimated 30%.

3.1.3 Chain-of-Thought (CoT) Prompting

Complex prompts included step-by-step reasoning instructions: "First, analyze the existing codebase. Second, list the files that need changes. Third, write each file." This reduced logic errors and ensured the AI considered edge cases before generating code.

3.1.4 Prompt Chaining

Instead of one monolithic prompt, development was split into 7 sequential phases, each with its own focused prompt (see Section 4). Each prompt consumed the output of the previous phase, enabling controlled, incremental progress without overwhelming the context window.

3.2 Prompt Structure Template

Vibe Profile

- Role: Senior Frontend Engineer + UI/UX Creative Director
- Aesthetic: Glassmorphism + Swiss minimalism with dark backgrounds
- Design Tokens: {colors, fonts, spacing, radii}

Product Brief

- Feature bullets (no paragraphs)
- Key user interactions
- Responsive requirements

Tech Rails

- Framework + version
- File tree structure
- Coding standards
- Anti-patterns to avoid

Design Tokens

- Color palette with hex values
- Typography (headings, body, code)
- Spacing scale
- Border radii
- Animation timings

3.3 Sample Prompts Used

3.3.1 Frontend Prompt (Excerpt)

You are a Senior Frontend Engineer + UI/UX Creative Director.

Vibe: Modern, sleek, glassmorphism + Swiss minimalism.

Dark backgrounds (#000000, #0A0A0A) with vibrant purple (#7C3AED) and cyan (#0891B2) accents. Inter font.

Build the complete OmniDraft React frontend with:

- Landing page (Hero with animated mode wheel, FAQs, CTA)
- Workspace (chat UI with SSE streaming, template picker, file upload)
- Auth page (magic link sign-in)
- History page (saved conversations)
- Settings page (account info)

Tech: React 18 + TypeScript + Vite + Tailwind CSS 3 + Framer Motion + Zustand

3.3.2 Backend Prompt (Excerpt)

You are a Senior Backend Engineer + Security Architect.

Build a FastAPI backend for OmniDraft that:

1. Streams tokens from NVIDIA LLM API via SSE
2. Authenticates users via Supabase JWT
3. Stores conversations and messages in Supabase PostgreSQL
4. Rate-limits at 20 requests/minute per IP
5. Supports file upload (PDF/TXT) for summarization mode

Tech: Python 3.11 + FastAPI + httpx + slowapi + supabase-py

Endpoints: POST /chat/stream, GET/POST/DELETE /conversations,
GET /templates, POST /upload, GET /export, POST /auth/callback

3.3.3 Database Prompt (Excerpt)

You are a Database Architect.

Design a PostgreSQL schema for OmniDraft with:

- profiles (id, email, created_at)
- conversations (id, user_id, mode, title, timestamps)
- messages (id, conversation_id, role, content, tokens_used)
- templates (id, mode, title, system_prompt, icon)

Enable Row-Level Security on user-owned tables.

Seed 12 templates: 4 Draft + 4 Summarize + 4 Creative.

4. Phase-by-Phase Development

Summary

Phase 1 — Frontend Foundation (Prompt 1)

Goal: Build the complete React frontend with all pages, components, routing, and state management.

Outcome: The AI generated a full multi-page SPA with:

- Landing page with HeroSection (animated OptionWheel), SupportedContentTypes, WhyOmniDraft, HowItWorks, FAQ accordion, and CTA
- Workspace with mode tabs (Draft/Summarize/Creative), chat message list, streaming text display, input bar, file upload zone, template picker, copy/export buttons
- Auth page with email-based magic link sign-in
- History page with conversation list and delete
- Settings page with account info and sign-out
- Zustand store for mode, conversation, and UI state
- Custom hooks (useAuth, useConversations, useStreamingChat)
- Glassmorphic dark UI with Tailwind utility classes

Phase 2 — Backend API (Prompt 2)

Goal: Build FastAPI backend with all routers, services, and middleware.

Outcome: The AI generated:

- 6 routers: auth, chat (SSE streaming), conversations, templates, upload, export
- NVIDIA LLM integration with streaming support, timeout, fallback model
- Supabase DB service (8 functions for conversation CRUD + profile management)
- JWT authentication middleware verifying Supabase tokens
- Rate limiting via slowapi (20 req/min per IP)
- Pydantic models for request/response validation
- Health check endpoints

Phase 3 — Database & Supabase Setup (Prompt 3)

Goal: Set up PostgreSQL schema with RLS policies and seed data.

Outcome: The AI generated:

- SQL migration for 4 tables: profiles, conversations, messages, templates
- Row-Level Security policies for user data isolation
- Indexes on user_id, conversation_id, and timestamps
- 12 seed templates with mode-specific system prompts
- Supabase Storage bucket for file uploads

Phase 4 — Docker Containerization (Prompt 4)

Goal: Package the full stack into a single container with Nginx reverse proxy.

Outcome: The AI generated:

- Multi-stage Dockerfile (3 stages: frontend build, backend deps, production image)
- Nginx configuration with SSE proxy support, gzip, security headers, SPA fallback
- Entrypoint script starting both uvicorn and nginx
- Docker Compose for local development (frontend + backend + hot reload)
- .dockerignore and non-root user setup

Phase 5 — AWS Deployment (Prompt 5)

Goal: Deploy the containerized application to AWS.

Outcome: The AI generated:

- AWS deployment scripts (ECR login, build, push)
- IAM policy document for deployer role (least-privilege)
- Dockerrun.aws.json for Elastic Beanstalk configuration
- apprunner.yaml for App Runner deployment
- Environment variables checklist for production

The application was successfully deployed to **AWS Elastic Beanstalk** at <http://omnidraft-prod.eba-pepktrup.us-east-1.elasticbeanstalk.com> running on a t3.micro instance with Docker AL2023 platform.

Phase 6 — Testing & Security (Prompt 6)

Goal: Add comprehensive test suite and security audit.

Outcome: The AI generated:

- pytest test suite with 5 test files (11 test cases)
- Fixtures for test client and auth headers
- Tests covering: streaming, auth, templates, conversations, LLM config
- Security checklist

Phase 7 — Final Polish (Prompt 7)

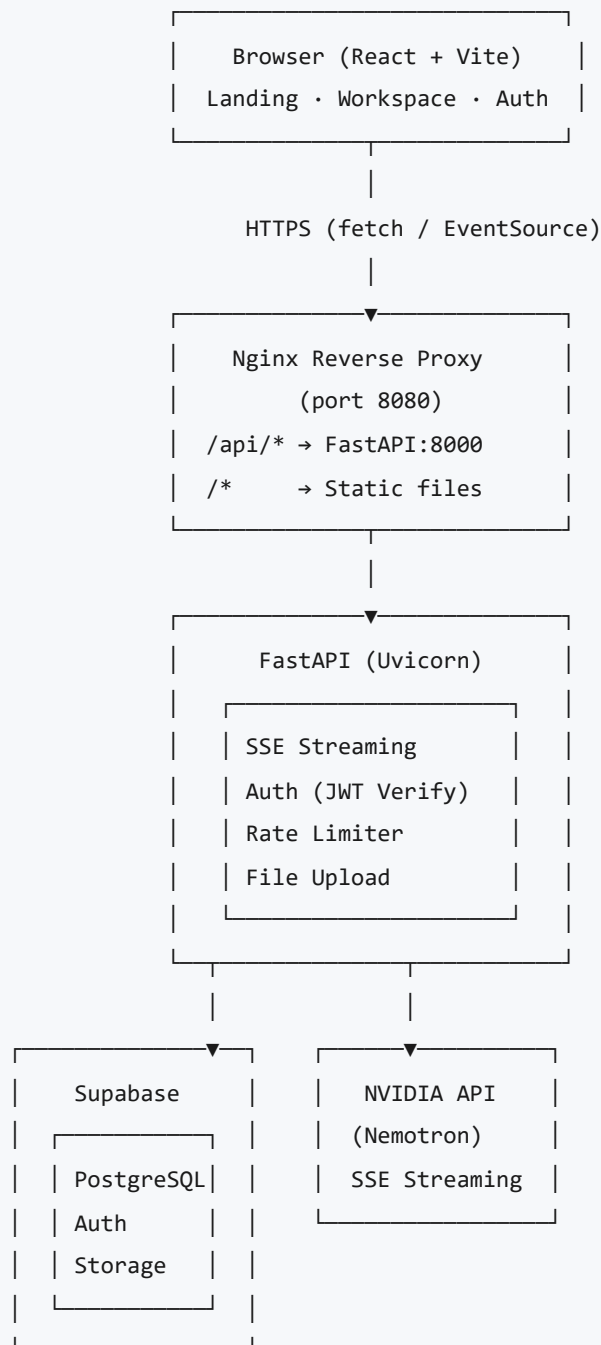
Goal: UI refinements, accessibility, performance optimization.

Outcome: The AI generated:

- Empty states, loading states, and error states for all components
- Skeleton loaders with shimmer animation
- Accessibility improvements (aria attributes, keyboard navigation, reduced motion)
- Framer Motion spring animations
- Toast notification system
- ErrorBoundary component

5. Application Architecture

5.1 High-Level Architecture Diagram



5.2 Data Flow

1. **User** enters prompt in React frontend
2. **Frontend** sends POST /api/chat/stream with mode + message
3. **FastAPI** middleware verifies Supabase JWT, applies rate limiting
4. **Router** creates/updates conversation in Supabase PostgreSQL, saves user message
5. **LLM Service** calls NVIDIA API with mode-specific system prompt + user message
6. **NVIDIA** streams tokens back via SSE
7. **FastAPI** proxies each token chunk to the frontend via yield
8. **React** renders tokens word-by-word using StreamingText with blur animation
9. **After completion**, full assistant message is saved to PostgreSQL
10. **User** can later view, export, or delete conversations from History

5.3 Frontend Route Map

Route	Component	Auth Required
/	Home (Landing Page)	No
/workspace	Workspace (Chat UI)	Yes
/history	Conversation History	Yes
/settings	Account Settings	Yes
/auth	Magic Link Sign-In	No

5.4 Backend API Endpoints

Method	Path	Description
GET	/health	Health check
POST	/api/auth/callback	Create/update user profile after auth
GET	/api/auth/me	Get current user profile
POST	/api/chat/stream	SSE streaming chat (rate limited)
GET	/api/conversations	List user's conversations
GET	/api/conversations/{id}	Get conversation with messages
DELETE	/api/conversations/{id}	Delete conversation

GET	/api/templates	List templates (optional ?mode filter)
POST	/api/upload	Upload file (max 10MB)
GET	/api/export/{id}	Export conversation (?format=txt md)

5.5 Database Schema

Table	Key Columns	RLS
profiles	id (PK, references auth.users), email, created_at, total_tokens_used	Own row only
conversations	id (PK), user_id (FK), mode, title, created_at, updated_at	Own rows only
messages	id (PK), conversation_id (FK), role, content, tokens_used, created_at	Via conversation owner
templates	id (PK), mode, title, description, system_prompt, icon, sort_order	Public read-only

6. Challenges Encountered & Resolutions

6.1 AWS App Runner Subscription Restriction

Challenge: The preferred deployment service (AWS App Runner) was unavailable on the student AWS account, returning a `SubscriptionRequiredException` error. App Runner requires a support subscription beyond the basic free tier.

Resolution: Migrated to **AWS Elastic Beanstalk** (Docker AL2023 platform, single-instance t3.micro) as the deployment target. Elastic Beanstalk offered equivalent functionality (container-based deployment, health monitoring, auto-scaling) without the subscription requirement. The same Docker image worked without modification.

6.2 Environment Variable Exposure in Git History

Challenge: During early development, `.env` files containing Supabase service keys and NVIDIA API keys were accidentally committed to the Git repository and pushed to GitHub. This created a security vulnerability with exposed credentials in version history.

Resolution: Ran `git filter-branch` to scrub all sensitive files from the entire Git history, removing `backend/.env` and `backend/init_db.py` (which contained inline keys). A `.gitignore` was created to prevent future leaks. The exposed keys were noted for rotation.

6.3 Supabase Auth Redirect Configuration

Challenge: After deployment to Elastic Beanstalk, the Supabase magic link auth flow redirected users to `localhost:5173` instead of the live URL, because the Supabase project settings had `site_url` and `uri_allow_list` configured only for local development.

Resolution: Updated the Supabase project's authentication settings via the Management API to include the Elastic Beanstalk URL (`http://omnidraft-prod.eba-pepktrup.us-east-1.elasticbeanstalk.com`) in both `site_url` and `uri_allow_list`. The frontend was also updated to use `window.location.origin` for the `emailRedirectTo` parameter, making it environment-agnostic.

6.4 SSE Streaming Proxy Buffering

Challenge: The initial Nginx configuration had proxy buffering enabled for SSE streams. This caused the streaming response to buffer in Nginx before being sent to the frontend, negating the real-time streaming benefit.

Resolution: Configured Nginx to disable proxy buffering specifically for the `/api/` location block (`proxy_buffering off;` , `proxy_cache off;` , `proxy_read_timeout 86400s;`), allowing SSE chunks to pass through immediately.

6.5 Stuck EB Environment on Initial Setup

Challenge: An earlier attempt at deployment created a stuck Elastic Beanstalk environment (`MyIBMWEBSTIE-OMNIDRAFT`) that failed to terminate and blocked creation of a new environment.

Resolution: Forcibly terminated the old environment via the AWS CLI (`aws elasticbeanstalk terminate-environment`) and cleaned up associated resources (EC2 instance, security groups). Created a fresh environment `omnidraft-prod` with proper configuration.

6.6 AI Hallucination: Imagined Library

Challenge: An early prompt generated code referencing a non-existent Python library for SSE handling. The AI had hallucinated a plausible-sounding library name from its training data.

Resolution: Added explicit constraints to all prompts: "Use only standard library + httpx for HTTP. Do not use any non-standard libraries not listed in the requirements." The SSE implementation was rewritten using FastAPI's native `StreamingResponse` with httpx async streaming.

7. Key Learnings & Reflection

7.1 Technical Learnings

- **Prompt quality determines output quality.** Investing 10 minutes to write a structured, detailed prompt (with role, goal, context, constraints, and examples) produced dramatically better first-pass code than a vague one-line request. The RGC (Role-Goal-Context) framework was consistently effective.
- **Chain-of-Thought reduces debugging.** Asking the AI to "think step by step" and "list the files that need changes before writing code" cut the iteration cycle by roughly half. The AI would surface edge cases and dependency issues before generating code, instead of producing broken code that required multiple fix cycles.
- **Context window management is critical.** Prompt chaining — splitting development into 7 sequential phases rather than one monolithic prompt — kept each prompt within the model's effective context window and prevented the AI from "forgetting" earlier instructions.
- **Containerization simplifies deployment.** The same Docker image that ran locally deployed to AWS Elastic Beanstalk without modification. The "works on my machine" problem was completely eliminated.
- **Security must be proactive, not reactive.** Accidentally committing secrets to Git taught the importance of `.gitignore` setup from day one. A single `.gitignore` at project root with `.env`, `*.pem`, and `*key*` patterns would have prevented the entire credential exposure incident.

7.2 Vibe Coding Reflection

Vibe Coding fundamentally changes the developer's role from "syntax writer" to "architect and creative director." The most valuable skills were not remembering framework APIs or language syntax, but:

- **Architectural thinking:** Designing the system structure, data flow, and component boundaries before prompting
- **Prompt crafting:** Writing precise, contextual, constraint-rich instructions
- **Verification:** Reviewing AI-generated code for correctness, security, and performance — the AI is a pattern-matcher, not a truth machine
- **Iteration management:** Feeding errors back to the AI as learning signals in the "crash and fix" loop

"You are no longer a syntax writer. You are an Architect, a Systems Thinker, a Creative Director." — IBM SkillsBuild Masterclass 1

The project demonstrated that a single developer with no prior FastAPI or Tailwind experience can ship a production-ready, containerized, cloud-deployed AI application in a matter of sessions — provided they master the art of communicating intent to an AI collaborator.

7.3 Future Improvements

- **HTTPS:** Add CloudFront or Let's Encrypt SSL for secure connections
- **RAG Mode:** Implement pgvector for retrieval-augmented generation over user-uploaded documents
- **Voice Input:** Integrate Web Speech API for hands-free interaction
- **Team Workspaces:** Add sharing and collaboration features
- **Custom Templates:** Allow users to create and save their own prompt templates
- **Cost Monitoring:** Configure AWS Budget alerts to prevent unexpected charges

8. Appendix: Complete Prompt Engineering Record

The following table summarizes all prompt engineering artifacts generated during development. Full prompt files are available in the `prompts/` directory of the repository.

File	Phase	Lines	Frameworks Used
PROMPT-01-FRONTEND.md	React Frontend	184	RGC, Markdown Spec, Design Tokens
PROMPT-02-BACKEND.md	FastAPI Backend	207	RGC, CoT, Tech Rails
PROMPT-03-DATABASE.md	Supabase DB Schema	193	Few-Shot (SQL examples), Spec Sheet
PROMPT-04-DOCKER.md	Docker Containerization	242	RGC, CoT, Anti-Patterns
PROMPT-05-AWS-DEPLOYMENT.md	AWS Cloud Deployment	215	Step-by-Step, Checklist
PROMPT-06-TESTING.md	Testing & Security	191	CoT, Acceptance Criteria
PROMPT-07-FINAL-POLISH.md	UI Polish & A11y	246	Vibe Profile, WCAG Checklist
PROMPT-MASTER-CONTINUATION.md	Master Orchestration	597	Prompt Chaining, Phase-by-Phase

8.1 Full Prompt File Listing

The complete prompt engineering artifacts are available at: `github.com/amalbyte-afk/OmniDraft/tree/master/prompts`

- `prompts/PROMPT-01-FRONTEND.md` — Complete React frontend build prompt with design tokens (184 lines)
- `prompts/PROMPT-02-BACKEND.md` — FastAPI backend with SSE streaming, auth, rate limiting (207 lines)
- `prompts/PROMPT-03-DATABASE.md` — PostgreSQL schema, RLS policies, seed data (193 lines)
- `prompts/PROMPT-04-DOCKER.md` — Multi-stage Docker, Nginx, Docker Compose (242 lines)

- `prompts/PROMPT-05-AWS-DEPLOYMENT.md` — ECR, App Runner, IAM, budgets (215 lines)
- `prompts/PROMPT-06-TESTING.md` — pytest suite, security checklist (191 lines)
- `prompts/PROMPT-07-FINAL-POLISH.md` — Animations, accessibility, grading checklist (246 lines)
- `prompts/PROMPT-MASTER-CONTINUATION.md` — Master prompt orchestrating phases 3-7 (597 lines)

8.2 Live Application

URL: <http://omnidraft-prod.eba-pepktrup.us-east-1.elasticbeanstalk.com>

GitHub Repository: <https://github.com/amalbyte-afk/OmniDraft>

— End of Report —